

Patent-Style Technical Specification

SuperMoon Prompt Analysis, Research Automation, Verification & PDF Generation Interface

Analysis ID	34f9dce09a261638	Prompt SHA-256	34f9dce09a261638725cbf6d...
Scientific readiness	93.3/100	Architecture score	93.8/100
Risk class	VERY_HIGH	CRI	0.665
SM32 runtime	ACTIVE	Generated	2026-08-19 19:43 UTC

Evidence discipline

This document distinguishes prompt-derived analysis from externally validated scientific results. No external solver, laboratory, patent novelty search, certification authority, or third-party qualification is represented as having been executed unless separately evidenced.

Notice

This is a patent-style technical drafting artifact, not a legal opinion, filing recommendation, novelty determination, freedom-to-operate analysis, or representation of patentability. A professional prior-art search and jurisdiction-specific counsel remain necessary before filing.

1. Technical Field

The disclosure relates to computer-implemented scientific research orchestration, prompt analysis, numerical-governance workflows, knowledge retrieval, evidence management, interactive three-dimensional research interfaces, and automated generation of scientific and patent-style PDF artifacts.

2. Background

Complex master prompts often combine objectives, mathematical methods, software requirements, risk-sensitive decision logic and reporting requirements in unstructured text. A technical challenge is converting that text into a traceable computational workflow while preventing unsupported claims from being treated as validated results.

3. Summary of the System

The disclosed system ingests a master prompt, computes a structural and governance analysis, queries a compressed indexed knowledge corpus, invokes an embedded SuperMoon 32 qualified research runtime for risk assessment where available, visualizes the workflow through a Babylon.js 9 interface, and produces evidence-disciplined scientific and patent-style reports.

4. System Architecture

A backend service provides prompt ingestion, knowledge retrieval, runtime bridging, computational risk assessment, analysis synthesis and PDF generation. A browser front end renders control panels and a three-dimensional research graph. A seekable compressed knowledge cache preserves the full canonical corpus while enabling chunk-level retrieval.

5. Knowledge Integration

The canonical SuperMoon corpus remains preserved as a compressed text artifact. During setup, a chunk store and compact SQLite conceptual-term metadata index are generated. Conceptual terms are indexed; exact identifier queries can fall back to canonical streaming search, reducing index inflation from large enumerated catalogs.

6. Analysis Pipeline

The analyzer detects domains, sections, goals, constraints, outputs, equation-like statements, risk triggers, evidence-strength claims, verification/validation/UQ coverage, reproducibility controls and reporting requirements. The output is represented as structured machine-readable data and a staged research workflow.

7. Risk-Aware Execution Governance

The system computes a Computational Risk Index through the embedded qualified runtime when available. The outcome determines whether external qualified tools, independent verification and human review are mandatory, recommended or optional.

8. Evidence and Claim Governance

High-strength assertions are separated from demonstrated results. Claims indicating certification, proof, guaranteed performance, absolute success or production readiness are flagged for evidence attachment or wording downgrade.

9. Three-Dimensional Interface

Babylon.js 9 renders an interactive research nucleus and workflow nodes representing formalization, model selection, risk, routing, verification, UQ, evidence and reporting. Visual state can be updated from backend analysis without

assigning scientific truth to visualization alone.

10. PDF Artifact Generation

The system generates a scientific report and patent-style technical specification directly from the structured analysis. Each artifact carries prompt hashes, analysis identifiers, risk state, limitations and a statement distinguishing prompt analysis from external validation.

11. Example Processing Sequence

- 01 — Ingest & normalize: Hash prompt, identify sections, constraints, claims and domain vocabulary.
- 02 — Problem formalization: Define objective, inputs, outputs, governing equations, assumptions, boundary/initial conditions and acceptance criteria.
- 03 — Dimensional & regime analysis: Check units, nondimensional groups, model validity and operating regime.
- 04 — Risk / scale assessment: Compute CRI and escalation policy. Current inferred class: VERY_HIGH (CRI=0.665).
- 05 — Method & solver routing: Select internal SM32 modules and, when required, qualified external tools.
- 06 — Execution plan: Build reproducible DAG, parameter sweeps, seeds, manifests, checkpoints and failure semantics.
- 07 — Independent verification: Require a second numerical/analytical path when risk policy indicates it.
- 08 — Validation: Compare against benchmark, experimental, reference, or acceptance data.
- 09 — UQ & sensitivity: Quantify numerical/parameter/model uncertainty and sensitivity.
- 10 — Discrepancy analysis: Classify solver disagreement and investigate mesh, model, tolerance, implementation and data causes.
- 11 — Evidence & provenance: Hash artifacts, track source lineage, limitations, known defects and claim level.
- 12 — Decision synthesis: Separate observed result, inference, hypothesis, limitation and recommendation.
- 13 — Scientific / patent PDF: Generate evidence-backed scientific report and patent-style technical specification.

12. Illustrative Claims

The following claims are drafting examples generated from the implemented architecture and should be reviewed against prior art and jurisdiction-specific requirements.

1. A computer-implemented system comprising: a prompt ingestion module configured to receive a scientific or engineering master prompt; a deterministic analysis engine configured to extract objectives, constraints, computational domains, evidence-sensitive claims and research-governance requirements; a knowledge retrieval engine configured to retrieve relevant chunks from a compressed indexed corpus; a risk engine configured to determine a computational risk state; and a report engine configured to generate a scientific PDF artifact from a structured analysis record.
2. The system of claim 1, wherein the knowledge retrieval engine stores chunk payloads in independently compressed blocks and stores chunk metadata and compact searchable conceptual-term summaries in a relational index.
3. The system of claim 1, wherein the risk engine invokes an embedded SuperMoon 32 qualified research runtime and returns at least a computational risk index, a risk class, an external-tool escalation state, an independent-verification requirement and a human-review requirement.
4. The system of claim 1, further comprising a three-dimensional browser visualization engine implemented with Babylon.js 9 and configured to visualize research workflow stages as interactive nodes whose state is updated from the structured analysis record.
5. The system of claim 1, wherein the deterministic analysis engine flags high-strength claims lacking referenced benchmark, validation, qualification, experimental or external-reproduction evidence.
6. The system of claim 1, wherein generation of a patent-style PDF and generation of a scientific PDF use the same prompt hash and analysis identifier to maintain traceability between source prompt and output artifacts.

7. The system of claim 1, wherein a fallback exact-search path scans the canonical compressed corpus for identifier-heavy queries excluded from the compact conceptual index.
8. A computer-implemented method comprising receiving a master prompt; normalizing the prompt; computing research-governance coverage; assessing computational risk; retrieving relevant knowledge chunks; creating an evidence-aware workflow; and generating at least one portable document format artifact carrying a cryptographic identifier of the received prompt.
9. The method of claim 8, further comprising preventing an externally unvalidated computational conclusion from being labeled as certified, experimentally validated, independently reproduced or qualification-acceptable solely by reason of generation by the system.
10. A non-transitory computer-readable medium storing instructions that, when executed, cause a computing device to perform the method of claim 8.

13. Prompt-Derived Embodiment Profile

Property	Observed / inferred value
Analysis ID	34f9dce09a261638
Scientific readiness	93.3
Patent drafting readiness	89.7
Risk class	VERY_HIGH
Detected domains	Scientific Research, HPC / Numerics, Statistics / UQ, ML / AI, Structures / FEA
Claims requiring evidence	7

14. Known Limitations

- The analyzer is deterministic and lexical/structural; it does not by itself establish scientific truth or patent novelty.
- The included external-tool registry does not imply certification of third-party solvers or hardware.
- Full-corpus knowledge integration improves retrieval and traceability but does not transform inherited text into verified evidence.
- Patent claims may require substantial narrowing, redrafting and prior-art differentiation before filing.
- Safety-critical or certification-sensitive decisions require the escalation paths identified by the risk engine.

Appendix A — Source Master Prompt Excerpt

```
===== BEGIN SUPER MOON 32.0 OMEGA QUALIFIED RESEARCH AUTOMATION VERIFICATION
ORCHESTRATION AUTHORITATIVE SUCCESSOR =====
<<<SM32Q_OUTER_MERGE_RECEIPT>>>
{
  "authoritative_runtime": "SM32 + supermoon32.qualified",
  "branch_coverage_percent": 98.17708333333333,
  "certification_acceptance": "NOT_CLAIMED",
  "completed_endurance_seconds": [
    10,
    15
  ],
  "created_unix": 1787160455.0988,
  "format": "SM32Q_RELEASE_RECEIPT_V1",
  "overlay_policy": "ALL_PREVIOUS_CODE_RETAINED_BYTE_FOR_BYTE; QUALIFIED_CORE_APPENDED;
LATEST_SM32Q_LAYER_AUTHORITATIVE_FOR_QUALIFIED_ORCHESTRATION",
  "previous_gzip_bytes": 87658945,
  "previous_gzip_sha256": "831cfccd92167b121b88e658408e2e2bbf6331a269c3cb005d3698dfb9178121",
  "previous_member_preserved_byte_for_byte": true,
  "qualification_status": "PASS_WITH_LIMITATIONS",
  "qualified_payload_manifest_entries": 48,
  "qualified_payload_manifest_sha256": "ee522a91fff0dfc67a4e329afea99a7423beea7fc5e173f129daf0a7b54118ca",
  "qualified_tests_passed": 130,
  "real_external_qualified_tool_execution": "NOT_EXECUTED_NO_TOOL_PROVIDED",
```

```

"release": "SUPER MOON 32.0 OMEGA \u2014 2014 QUALIFIED RESEARCH AUTOMATION VERIFICATION ORCHESTRATION CORE",
"sm32q_file_count": 50,
"statement_coverage_percent": 99.58847736625515,
"tests_total_passed": 323
}
<<<END_SM32Q_OUTER_MERGE_RECEIPT>>>
<<<SM32Q_FILE path="README_SM32_QUALIFIED_CORE.md"
sha256="21a41d9df5c1ac3deff2494e6fca38e550afb8c340a1e15e6fd0d1bab966b917" bytes="1230">>>
# SUPER MOON 32.0 OMEGA — Qualified Research, Automation, Verification & Orchestration Core

```

This additive SM32 successor layer implements the qualified research/orchestration master prompt without deleting or rewriting the prior SUPER MOON 32.0 Full Implementation Dominion payload.

The layer adds executable risk-aware computation, mandatory escalation policy, qualified-tool registry and CLI adapter mechanics, solver routing, mathematical problem formalization, dimensional/regime analysis, model-validity checking, independent verification/discrepancy analysis, GCI/Richardson tools, advanced UQ/sensitivity/reliability, adjoint/POD/DMD/Bayesian research math, DOE/surrogates/multi-fidelity helpers, DAG orchestration, provenance/evidence/claim governance, qualification and change-impact ledgers, HPC routing/scaling metrics, multi-solver normalization, co-simulation, precision governance, domain-specific routing policies, benchmark management and endurance management.

Truth policy: this code does **not** claim that any real external CAE/HPC/CAD/ML vendor tool has been certified or executed. External-tool routing mechanics are implemented and tested; real tool qualification remains tool/version/context specific.

```

<<<END_SM32Q_FILE>>>
<<<SM32Q_FILE path="docs/SM32Q_MASTER_PROMPT_EN.txt"
sha256="947983489d19ac80dc918e73ba5d0772fdea9880dd2bb80ff329196afbb0cc77" bytes="61257">>>
SUPER MOON — QUALIFIED RESEARCH, AUTOMATION, VERIFICATION & ORCHESTRATION CORE
ADVANCED MATHEMATICS, MULTI-SOLVER GOVERNANCE, RISK-AWARE COMPUTATION,
QUALIFIED-TOOL ESCALATION, VALIDATION, UQ, HPC & EVIDENCE MASTER PROMPT

```

```

=====
0. PRIME DIRECTIVE
=====

```

You are designing the next evolution of SUPER MOON as a high-assurance:

RESEARCH + AUTOMATION + VERIFICATION + ORCHESTRATION CORE

The system is NOT required to replace every mature engineering, scientific, certification, HPC, CAD, CAE, AI, or regulatory tool.

Its primary role is to:

1. formulate complex scientific and engineering problems correctly;
2. formalize their governing mathematics;
3. select suitable numerical and computational methods;
4. evaluate risk, criticality, uncertainty, and scale before execution;
5. execute internal SUPER MOON solvers where appropriate;
6. invoke qualified external tools when risk, scale, certification, or safety makes external qualification mandatory;
7. execute independent verification paths;
8. compare solver outputs;
9. quantify uncertainty and numerical error;
10. detect disagreements and investigate their cause;
11. automate parameter studies, design-space exploration, validation, optimization, and reproducibility;
12. produce evidence-backed technical conclusions;
13. maintain strict provenance, auditability, and claim discipline.

Target architecture:

```

PROBLEM
↓
FORMALIZATION
↓
MATHEMATICAL MODEL
↓
RISK / SCALE / CRITICALITY ASSESSMENT
↓
METHOD SELECTION

```

↓
 INTERNAL SOLVER / QUALIFIED EXTERNAL TOOL / BOTH
 ↓
 AUTOMATED EXECUTION
 ↓
 INDEPENDENT VERIFICATION
 ↓
 VALIDATION
 ↓
 UNCERTAINTY QUANTIFICATION
 ↓
 DISCREPANCY ANALYSIS
 ↓
 REPRODUCIBILITY
 ↓
 EVIDENCE PACKAGE
 ↓
 ENGINEERING / RESEARCH DECISION

Defining principle:

NO CRITICAL DECISION FROM A SINGLE UNVERIFIED COMPUTATIONAL PATH.

1. CORE DESIGN PHILOSOPHY

Do not maximize:

- number of algorithms;
- number of classes;
- number of modules;
- number of solver names;
- number of architecture diagrams;
- number of unsupported theoretical claims.

Maximize:

- correctness of problem formulation;
- mathematical rigor;
- solver suitability;
- numerical stability;
- verification depth;
- validation quality;
- uncertainty awareness;
- solver independence;
- reproducibility;
- traceability;
- evidence quality;
- defensibility of decisions.

Quality != FeatureCount

Quality = f(Correctness, Verification, Validation, Uncertainty,
 Reproducibility, Traceability, Qualification, Evidence)

2. THREE-TIER COMPUTATIONAL ARCHITECTURE

LEVEL A — INTERNAL COMPUTATIONAL CORE

Use SUPER MOON internal implementations for research, prototyping, algorithm development, solver experimentation, comparative studies, low/medium-scale simulations, hypothesis testing, sensitivity analysis, reduced-order models, design-space exploration, surrogate construction, and independent cross-checking.

LEVEL B — QUALIFIED TOOL ORCHESTRATION

Use externally qualified or mature tools when scale exceeds validated internal capacity; human safety, formal certification, airworthiness, production HPC,

industrial CAD robustness, extreme distributed execution, or formal independent authority review requires them.

LEVEL C — MULTI-SOLVER VERIFICATION

For high-risk problems require combinations such as:

SUPER MOON internal solver

+

qualified solver A

+

qualified solver B

+

analytical / manufactured / experimental reference

Never assume numerical agreement merely because tools are reputable.

3. COMPUTATIONAL RISK INDEX

Build a formal Computational Risk Index (CRI).

Define normalized variables in [0,1]:

C = criticality

S = computational scale

U = uncertainty

I = consequence of incorrect result

E = evidence deficiency

N = novelty of modeled regime

Q = qualification deficiency

$$CRI = wC*C + wS*S + wU*U + wI*I + wE*E + wN*N + wQ*Q$$

with $\text{sum}(\text{weights}) = 1$.

Suggested classes:

$CRI < 0.20$ LOW

$0.20 \leq CRI < 0.40$ MODERATE

$0.40 \leq CRI < 0.60$ HIGH

$0.60 \leq CRI < 0.80$ VERY_HIGH

$CRI \geq 0.80$ CRITICAL

Weights must be domain-specific rather than universal constants.

4. MANDATORY ESCALATION CONDITIONS

Qualified external tools become mandatory when any of the following applies:

- human safety;
- life-critical engineering;
- regulatory certification;
- airworthiness approval;
- nuclear safety;
- critical infrastructure;
- medical-device regulatory decisions;
- flight-critical structural verification;
- extreme nonlinear structural analysis;
- very large production CFD;
- billion-cell or comparable distributed CFD;
- extreme sparse distributed systems;
- industrial CAD exchange requiring production-kernel robustness;
- production GPU training beyond validated internal capability;
- formal contractual/legal acceptance criteria;
- independent authority review.

SUPER MOON may still provide research analysis, preprocessing, mesh studies, independent verification, sensitivity analysis, optimization proposals, surrogate models, post-processing, uncertainty analysis, discrepancy analysis, and evidence organization.

But internal output alone must NOT be represented as final authority unless qualification evidence explicitly supports that status.

5. QUALIFIED TOOL REGISTRY

Create QualifiedToolRegistry records containing:

tool_name
 vendor_or_project
 version
 domain
 solver_types
 supported_equations
 verified_use_cases
 known_certification_contexts
 supported_hardware
 validated_scale_range
 input_formats
 output_formats
 API
 CLI
 automation_interface
 license
 qualification_level
 validation_evidence
 benchmark_evidence
 known_limitations
 known_failure_modes
 supported_operating_systems
 dependency_requirements
 reproducibility_notes

6. QUALIFIED TOOL LEVELS

Q0 = UNKNOWN
 Q1 = AVAILABLE
 Q2 = TESTED
 Q3 = BENCHMARKED
 Q4 = INDUSTRIALLY VALIDATED
 Q5 = CERTIFICATION-ACCEPTABLE FOR SPECIFIC CONTEXT

Never convert Q1 into Q5 through marketing language.
 Qualification must remain use-case-specific.

7. INTERNAL CAPABILITY MATURITY

M0 = CONCEPT
 M1 = IMPLEMENTED
 M2 = UNIT_TESTED
 M3 = NUMERICALLY_VERIFIED
 M4 = BENCHMARKED
 M5 = STRESS_TESTED
 M6 = EXTERNALLY_REPRODUCED
 M7 = INDUSTRIALLY_VALIDATED
 M8 = QUALIFIED FOR DEFINED CONTEXT

No internal capability may be silently treated as M8.

8. SOLVER ROUTING ENGINE

Create SolverRoutingEngine.

Inputs:

- governing equations;
- dimensionality;
- physics;
- geometry;
- mesh type;
- scale;
- stiffness;
- nonlinearity;
- expected conditioning;
- uncertainty;
- accuracy target;
- runtime budget;
- memory budget;
- hardware;
- criticality;
- certification requirement;
- reproducibility requirement;
- external validation requirement.

Outputs:

- recommended internal solver;
- recommended qualified external solver;
- independent verification solver;
- recommended discretization;
- expected numerical risks;
- validation strategy;
- required evidence;
- escalation decision;
- confidence target.

9. SOLVER SELECTION OBJECTIVE

For candidate solver j:

$$J_j = w_1 * \text{ExpectedError}_j + w_2 * \text{Runtime}_j + w_3 * \text{MemoryCost}_j + w_4 * \text{QualificationRisk}_j + w_5 * \text{ModelMismatch}_j + w_6 * \text{ReproducibilityRisk}_j$$

Use Pareto analysis where objectives conflict.

Minimize:

error, runtime, memory, monetary cost, qualification risk, uncertainty,
model-form error

subject to:

Accuracy $\geq$ required_accuracy
Qualification $\geq$ required_qualification
Memory $\leq$ available_memory
Runtime $\leq$ allowed_runtime
Evidence $\geq$ required_evidence

=====

10. PROBLEM FORMALIZATION ENGINE

=====

No solver may execute before creating:

ProblemDefinition
DomainDefinition
UnknownVariables
GoverningEquations
InitialConditions
BoundaryConditions
MaterialModels
ConstitutiveModels
Constraints
Objectives
UncertainParameters
Observables
ValidationTargets
AcceptanceCriteria
Units
CoordinateSystems
ReferenceFrames
Assumptions
ValidityDomain

=====

11. EQUATION DEPENDENCY GRAPH

=====

Represent mathematical dependencies as $G=(V,E)$.

Vertices:

variables, fields, equations, constraints, constitutive laws, parameters,
solvers, observables, datasets, experiments.

Edges:

dependency, coupling, causality assumption, data flow, discretization
dependency, solver dependency.

Use the graph for missing-variable detection, coupling analysis,
circular-dependency detection, and verification planning.

=====

12. DIMENSIONAL ANALYSIS ENGINE

=====

Verify $[LHS] = [RHS]$ for every equation where dimensional information exists.

Detect:

SI/imperial mismatch
m/mm mismatch
Pa/MPa mismatch
kg/g mismatch
s/ms mismatch
rad/degree mismatch
inconsistent derived units

Abort high-risk workflows on unresolved dimensional inconsistency.

=====

13. NONDIMENSIONALIZATION ENGINE

=====

Identify useful nondimensional groups where applicable:

Reynolds, Mach, Prandtl, Peclet, Courant, Froude, Knudsen, Biot,

Fourier, Strouhal, Rayleigh, Grashof, Weber.

Use them to identify regime, route solvers, assess assumptions, choose discretization, and define stability limits.

14. MODEL REGIME DETECTION

Detect operating regime rather than blindly applying one model.

CFD:

incompressible, weakly compressible, compressible, transonic, supersonic, hypersonic, laminar, transitional, turbulent, continuum, rarefied.

Structures:

linear, geometrically nonlinear, materially nonlinear, static, transient, modal, buckling, contact-dominated.

Heat transfer:

steady, transient, conduction, convection, radiation, conjugate.

15. MODEL VALIDITY DOMAINS

Every model must declare:

valid_parameter_range
assumptions
unsupported_conditions
expected_error_range
required_input_quality
mesh_requirements
time_step_requirements
stability_constraints
known_failure_modes

Prevent silent extrapolation beyond validity domains.

16. VERIFICATION VS VALIDATION

VERIFICATION:

Are the equations being solved correctly?

VALIDATION:

Do the equations adequately represent the real system?

Every project must carry independent statuses for both.

17. METHOD OF MANUFACTURED SOLUTIONS

Choose exact artificial solution $u^*(x,t)$, compute $L(u^*) = f^*$, solve numerically, and measure $e_h = \|u_h - u^*\|$.

Report L1, L2, and L-infinity error where appropriate. Verify expected convergence order.

18. ORDER OF ACCURACY

For refinement ratio r :

$$p = \log(e_h/e_{h/2}) / \log(2)$$

Compare observed p with theoretical order.

Do not call a method second-order based on implementation intent alone.

=====

19. GRID CONVERGENCE INDEX

=====

Automate coarse/me